

Unidad N° 4: Interacción con la base de datos

Programación Orientada a Objetos – ISAM 2025

Resumen de la unidad

- Trabajaremos principalmente con las carpetas database y app.
- **Migraciones:** definir estructura de la BD.
- **Seeders y Factories:** definir datos de prueba en las tablas.
- **Modelos:** enlazar cada tabla de la BD a la aplicación.
- **Eloquent:** realizar las consultas SQL.
- **CRUD completo.**

Laravel y BD

- Al instalar Laravel le indicamos cual base de datos usar.
- Elegimos SQLite, la base de datos se encuentra en el archivo `database/database.sqlite`.
- Si bien SQLite es sencillo, usaremos MySQL/MariaDB para una mejor gestión del DBMS.
- Para ello debemos cambiar el tipo de conexión y las credenciales.

Credenciales de la BD

- En el archivo `.env` debemos indicar cuales son las credenciales de nuestra BD.

`DB_CONNECTION=mysql`

`DB_HOST=127.0.0.1`

`DB_PORT=3306`

`DB_DATABASE=gastos_personales`

`DB_USERNAME=root`

`DB_PASSWORD=`

Migraciones

- Mecanismo que permite definir la estructura de la base de datos dentro de la aplicación.
- Ventajas:
 - Permite conservar la estructura de la BD junto con el código.
 - Facilita la creación de la BD si portamos el proyecto a otra computadora u servidor.
 - En el caso de que suceda algo con nuestra base de datos, al tener la migración como respaldo evitamos tener que crear todo desde cero.

Migraciones en Laravel

- Para crear una migración utilizamos el siguiente comando:

```
php artisan make:migration create_nombre_tabla_table
```

- El archivo creado se guarda dentro de la carpeta `database/migrations`
- **Importante:** primero se deben crear las migraciones de tablas sin claves foráneas, dado que las migraciones se ejecutan en el orden que fueron creadas.

Estructura de una migración

- Al crearse, una migración cuenta con dos métodos:
 - **up()**: define la estructura de la tabla y se llama cuando ejecutamos la migración desde la consola. Como resultado, se creará o modificará la tabla en nuestra base de datos.
 - **down()**: define el proceso inverso al método up(), por ejemplo, la destrucción de la tabla, y se llama cuando hacemos un “rollback”.

Ejecución de migraciones

- Laravel crea una tabla especial llamada **migrations** en la cual registra todas las migraciones ejecutadas en grupos o “*batches*”.
- Para ejecutar una migración usamos el comando:
`php artisan migrate`
- Para deshacer una migración usamos el comando:
`php artisan migrate:rollback`
- Para recrear completamente la base de datos desde cero usamos el comando:
`php artisan migrate:fresh`

Ejemplo: tabla “transacciones”

- Por convención, las tablas se crean con nombres en plural.
- **NOTA:** según el diagrama ER del sistema, transacciones tiene claves foráneas. De momento las ignoraremos y más adelante corregiremos la migración de esta tabla.

transaction		
INT	id	PK
INT	user_id	FK
INT	category_id	FK
INT	group_id	FK
VARCHAR	description	
DECIMAL	amount	
DATE	transaction_date	
TIMESTAMP	created_at	
TIMESTAMP	updated_at	

1) Crear la migración

- El nombre de la migración puede ser cualquier, pero se sugiere que se llame *“create_nombre_tabla_table”*

```
php artisan make:migration create_transacciones_table
```

2) Definir la estructura de la tabla en la migración

```
Schema::create('transacciones', function (Blueprint $table) {  
    $table->id();  
    $table->string('descripcion');  
    $table->decimal('monto');  
    $table->date('fecha_transaccion');  
    $table->timestamps();  
});
```

3) Ingresar el comando para ejecutar las migraciones pendientes

- Una vez definida la estructura, aun no existe la tabla en la base de datos.
- Debemos ejecutar las migraciones pendientes con el comando:
`php artisan migrate`

Tipos de columnas

- **id()**: crea un campo INTEGER como clave primaria auto-incremental.
- **integer()**: crea un campo INTEGER común.
- **date()** o **dateTime()**: crea campos DATE o DATETIME.
- **decimal()** o **double()**: crea campos DECIMAL o DOUBLE.
- **timestamps()**: crea los campos especiales *created_at* y *updated_at* del tipo TIMESTAMP.
- **softDeletes()**: crea el campo especial *deleted_at* del tipo TIMESTAMP.
- **string()**: crea un campo VARCHAR.
- **text()**: crea un campo TEXT

Modificadores de columnas

- **after('columna')**: permite agregar un campo después de una columna existente. Es el contrario al método **before('columna')**.
- **default(\$valor)**: define un valor por defecto para la columna.
- **nullable()**: permite que en el campo se ingresen valores NULL.
- **unsigned()**: define que los valores numéricos del campo no tengan signos.

Índices

- **primary()**: crea un índice primario en el campo.
- **unique()**: crea un índice con valores únicos.
- **index()**: crea un índice común.
- **fullText()**: crea un índice de tipo full text.

Convenciones de Laravel

- Las tablas deben nombrarse de forma plural y si constan de más de una palabra serán separadas por guiones bajos.
- Laravel espera que el índice primario se llame ***id*** y que sea auto incremental.
- Si usamos el método `timestamps()` se agregarán dos campos especiales:
 - ***created_at***: se guarda automáticamente la fecha de creación de un registro.
 - ***updated_at***: se guarda automáticamente la fecha de última edición de un registro.
- Si usamos el método `softDeletes()` se agregará el campo ***deleted_at*** que permite implementar el borrado lógico sobre un registro.

Más información

- Migraciones: <https://laravel.com/docs/12.x/migrations>
- Tipos de columnas disponibles:
<https://laravel.com/docs/12.x/migrations#available-column-types>
- Modificadores de columnas:
<https://laravel.com/docs/12.x/migrations#column-modifiers>

¿MIGRACIONES?



¡YA MUESTRANOS COMO HACER CONSULTAS SQL!

Eloquent

- Es el ORM (Object Relational Mapper) utilizado por Laravel para vincular las tablas de la base de datos con los datos de la aplicación.
- Funcionalidades principales:
 - Mapeo de tablas mediante los Modelos.
 - Interfaz de consultas SQL mediante la sintáxis de querys.
 - Configuración de comportamiento mediante casts, mutadores, etc.

Modelos

- Son clases especializadas que replican la estructura de una tabla de la base de datos.
- Por defecto, Eloquent lee automáticamente todos los campos de una tabla y los convierte en atributos de la clase modelo.
- Además podemos definir las relaciones de las tablas directamente en los modelos.
- Todos los modelos de nuestra aplicación están en la carpeta `app/Models`

Crear el modelo Transaccion

- Para crear el modelo desde la consola ejecutamos el comando:
`php artisan make:model Transaccion`
- El modelo será creado en la carpeta *app/Models*.
- Una vez creado, podemos ejecutar consultas en la aplicación mediante el modelo.

Propiedades clave

- `protected $table = 'mis_transacciones';`
- `protected $primaryKey = 'transaccion_id';`
- `public $incrementing = false;`
- `protected $keyType = 'string';`
- `public $timestamps = false; // Si no vamos a utilizar los campos created_at y updated_at`
- `protected $guarded = []; // Deshabilitar protección de asignación masiva`

Leer los registros de una tabla

- El método `all()` nos permite realizar un “`SELECT * FROM tabla`”, devolviendo todos los registros.

```
$transacciones = Transaccion::all();
```

- El método `where()` nos permite aplicar una condición al `SELECT`. Si optamos por usarlo debemos encadenar el método `get()` al final.

```
$transacción = Transaccion::where("id", 2)->get();
```

Obtener 1 resultado/registro

- El método *find(id)* busca un registro único en base a su id primario.

```
Transaccion::find(1);
```

- Como alternativa, podemos utilizar *where()* con la condición a buscar y al final *first()* para obtener solamente el primer registro en base al criterio.

```
Transaccion::where("id", 1)->first();
```


Insertar un registro

- La forma más fácil de crear un nuevo registro es utilizar el método *create()* junto a un arreglo asociativo con los datos deseados.

```
Transaccion::create([  
    'descripcion' => 'Pago de telefonía móvil',  
    'monto' => 20000,  
    'fecha_transaccion' => date("Y-m-d"),  
]);
```

- **Nota:** tanto el id primario, como los campos `created_at` y `updated_at` se rellenan automáticamente.

Actualizar un registro

- Para actualizar un registro podemos utilizar el método `update()` – siempre teniendo en cuenta que debemos realizar un `where` previamente.

```
Transaccion::where('id', 1)
->update([
    'descripcion' => 'Compra de supermercado',
]);
```

Eliminar un registro

- Se puede eliminar un registro mediante su clave primaria usando el método *destroy()*

```
Transaccion::destroy(1);
```

- O también utilizando un *where()*

```
Transaccion::where('id', 1)->delete();
```